

LLM Evaluation & Testing

Proving your AI system actually works — evaluation datasets, golden datasets, LLM-as-a-judge, benchmarking, core quality metrics, hallucination/RAG/agent evaluation, prompt regression testing, and error analysis.

DIFFICULTY LEVEL

Intermediate

ESTIMATED PRACTICE TIME

100-130 minutes

ESTIMATED READING TIME

60-75 minutes

PREREQUISITES

Modules 01-15 (Prompt Engineering Mastery Bucket)

LEARNING OUTCOMES

By the end of this module, you will be able to: build evaluation and golden datasets; use LLM-as-a-judge alongside automated and human evaluation; benchmark systems on accuracy, relevance, faithfulness, and groundedness; evaluate hallucinations, RAG pipelines, and agents specifically; set up prompt regression testing; and run structured error analysis on failures.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Evaluation Basics
3.2	Evaluation Datasets, Golden Datasets & Test Cases
3.3	LLM-as-a-Judge
3.4	Automated Evaluation vs. Human Evaluation
3.5	Benchmarking
3.6	Core Evaluation Metrics (Accuracy, Relevance, Faithfulness, Groundedness)
3.7	Hallucination Evaluation
3.8	RAG Evaluation
3.9	Agent Evaluation
3.10	Prompt Regression Testing
3.11	Error Analysis
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

Module 09 introduced prompt-level testing and evaluation. This module goes much further — full evaluation systems for entire AI applications, including RAG pipelines and agents, with rigorous datasets, multiple evaluation methods, and structured failure analysis.

Why is it important?

An AI system without real evaluation is a system you're hoping works, not one you know works. Evaluation is what turns "it seemed fine when I tried it" into a defensible, measurable claim about quality — essential the moment an AI feature matters to real users.

"I Think It Works"



Systematic Evaluation



"I Can Prove It Works"

Why should AI Engineers learn it?

Evaluation is what separates AI engineering from casual prompting. It's how you catch regressions before users do, compare architectural choices with evidence (RAG design, Module 07; agent design, Module 08), and give stakeholders a real quality number instead of a vibe.

Where is it used in the real world?

Every mature AI product team runs evaluation pipelines before shipping prompt or model changes — this is the AI-engineering equivalent of automated test suites in traditional software.

Why is it relevant today?

As AI systems become more complex (agents, Module 08; RAG, Module 07) and more consequential (Module 10), "eval culture" has become a defining trait of serious AI engineering teams in 2025-2026 — this module is foundational, not optional, going forward.

Note

This module extends Module 09's prompt testing/evaluation directly. If Module 09 felt shaky, a quick revisit will make this module land much better.

2. Before You Begin

RECAP

Prompt testing & evaluation (Module 09, 3.1-3.2) — this module scales those same core ideas (test sets, defined criteria) up to full applications, RAG pipelines, and agents.

RECAP

Hallucinations (Module 01, 3.11) and **RAG** (Module 07) — both get their own dedicated, systematic evaluation treatment in this module.



Tip

You don't need a huge evaluation dataset to start. Even 20-30 well-chosen examples, evaluated consistently, beats zero systematic evaluation — start small and grow it over time.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

3.1 Evaluation Basics

DEFINITION

LLM evaluation is the systematic process of measuring how well an AI system performs its intended task, using defined data, metrics, and methods rather than informal impressions.

SIMPLE EXPLANATION

At its core, every evaluation setup answers three questions: what data do we test on (Section 3.2), what method do we use to score it (Sections 3.3-3.4), and what metric defines "good" (Section 3.6) — get all three deliberate, and you have a real evaluation system.

💡 *Like a school's grading system — you need the exam questions (data), someone or something to grade them (method), and a rubric defining what counts as a correct answer (metric). Missing any piece, and grades become meaningless.*

EVERYDAY EXAMPLE

Evaluating a summarization feature means deciding: which documents to test on, whether a human or another LLM scores the summaries, and what makes a summary "good" (concise? faithful? complete?).

WHY IT MATTERS

Without this three-part structure, "evaluation" collapses into someone eyeballing a few outputs — not reliable enough to base real decisions on.

COMMON MISTAKES

Jumping straight to a metric or tool without first deciding what data and method actually fit the task.

BEST PRACTICES

Define data, method, and metric together, deliberately, before building or running any evaluation.

SUMMARY

Evaluation basics come down to three deliberate choices — data, method, metric — the skeleton every other concept in this module fills in.

3.2 Evaluation Datasets, Golden Datasets & Test Cases

DEFINITION

An evaluation dataset is a set of inputs (and often expected outputs) used to test a system; a golden dataset is a especially high-quality, carefully verified subset used as a trusted reference standard; a test case is one single input-expected-output pair within either.

SIMPLE EXPLANATION

This builds directly on Module 09's test sets, formalized: many test cases make a dataset, and a dataset whose expected answers have been carefully verified (often by domain experts) becomes "golden" — trustworthy enough to measure against with confidence.

💡 *Like an answer key for a test — a large dataset is the full question bank, a golden dataset is the subset where you're absolutely certain the answer key is correct, and each individual question is a test case.*

EVERYDAY EXAMPLE

A legal document Q&A system's golden dataset might be 50 questions with answers verified by an actual lawyer, versus a broader dataset of 500 auto-generated question-answer pairs of less certain accuracy.

WHY IT MATTERS

Evaluation results are only as trustworthy as the dataset they're measured against — a flawed or unverified dataset produces misleading scores, regardless of how sophisticated the evaluation method is.

COMMON MISTAKES

Using an unverified, auto-generated dataset as if it were golden-quality, without any human review of its correctness.

BEST PRACTICES

Invest real verification effort into a small golden dataset rather than a large but shaky one; grow it incrementally as you find new important cases (including from Section 3.11's error analysis).

SUMMARY

Datasets, golden datasets, and test cases scale Module 09's testing concept into a formal, trustworthy measurement foundation.

3.3 LLM-as-a-Judge

DEFINITION

LLM-as-a-judge is using a separate LLM call to evaluate and score another LLM's output against defined criteria, instead of relying solely on exact-match rules or human review.

SIMPLE EXPLANATION

For subjective qualities that are hard to check with simple rules (Module 04's exact-match/format validation) — tone, helpfulness, coherence — you can prompt a strong model with the output and a rubric, and have it return a score or pass/fail judgment.

System Output



Judge LLM + Rubric



Score / Verdict

Like hiring an experienced editor to grade essays against a rubric, at a scale no single human reviewer could sustain — not perfect, but far more consistent and scalable than nothing.

EVERYDAY EXAMPLE

Scoring 500 customer support replies for "empathy" (1-5) using a judge-LLM prompt with the rubric and a few calibration examples, rather than reading all 500 by hand.

WHY IT MATTERS

LLM-as-a-judge unlocks evaluation at a scale human review can't sustain, for exactly the subjective qualities that simple automated checks can't capture.

COMMON MISTAKES

Trusting judge-LLM scores blindly without periodically validating them against human judgment — the judge itself can be biased or inconsistent.

BEST PRACTICES

Give the judge a clear, specific rubric (ideally with calibration examples); periodically spot-check judge scores against human review to confirm they're tracking well.

SUMMARY

LLM-as-a-judge scales subjective evaluation using another LLM call — powerful, but needs periodic human calibration to stay trustworthy.

3.4 Automated Evaluation vs. Human Evaluation

DEFINITION

Automated evaluation uses code or another model (like LLM-as-a-judge) to score outputs without a person in the loop; human evaluation relies on people directly reviewing and judging outputs.

SIMPLE EXPLANATION

Neither is strictly better — automated evaluation scales cheaply and runs continuously (great for regression testing, Section 3.10), while human evaluation catches nuance and edge cases automated methods miss, at a much higher cost per example.

 *Like automated factory quality-control sensors versus a skilled human inspector — the sensors catch obvious defects at massive scale cheaply; the human inspector catches subtle problems the sensors were never designed to notice.*

EVERYDAY EXAMPLE

Automated format/schema validation (Module 04) runs on every single production request; a human reviews a random sample of 50 conversations weekly for tone and edge-case quality.

WHY IT MATTERS

Relying on only one type leaves a real gap — automated-only misses nuance, human-only doesn't scale to catch every regression.

COMMON MISTAKES

Relying exclusively on automated evaluation for genuinely subjective, high-stakes qualities that really need human judgment.

BEST PRACTICES

Combine both: automated evaluation for continuous, cheap coverage; periodic human evaluation for nuance, calibration, and high-stakes spot-checks.

SUMMARY

Automated and human evaluation are complementary, not competing — the strongest systems use both deliberately.

3.5 Benchmarking

DEFINITION

Benchmarking is comparing your system's evaluation results against a fixed reference point — a previous version, a competing approach, or a public standard.

SIMPLE EXPLANATION

A raw evaluation score (e.g., "82% accuracy") means little on its own; benchmarking gives it meaning by comparing it against something — last month's score, a different model (Module 11), or a published standard.

 *Like a runner's race time meaning little in isolation, but becoming meaningful when compared against their own personal best, a competitor's time, or the world record.*

EVERYDAY EXAMPLE

Benchmarking your RAG system's accuracy against both its previous version (did the new chunking strategy help, Module 07, 3.2?) and a public RAG benchmark dataset for external context.

WHY IT MATTERS

Benchmarking connects evaluation scores to actual decisions — is this change worth shipping? Is this model worth its cost (Module 09, 3.4)?

COMMON MISTAKES

Comparing scores across evaluations that used different datasets or methods, producing an apples-to-oranges comparison that looks meaningful but isn't.

BEST PRACTICES

Keep the dataset and method fixed when comparing across versions/models — only the system under test should change between benchmark runs.

SUMMARY

Benchmarking turns a raw evaluation score into a meaningful comparison — the step that actually informs real engineering decisions.

🔗3.6 Core Evaluation Metrics: Accuracy, Relevance, Faithfulness, Groundedness

DEFINITION

Accuracy measures whether an output is factually/logically correct; relevance measures whether it actually addresses the question asked; faithfulness measures whether it's consistent with a given source; groundedness measures whether every claim can be traced back to supporting evidence.

SIMPLE EXPLANATION

These four metrics answer different questions and can diverge — an answer can be relevant but inaccurate, or accurate but not grounded in the provided context (meaning the model "got lucky" from trained knowledge rather than the actual source, Module 07).

Metric	Question It Answers
Accuracy	Is this correct?
Relevance	Does this address the actual question?
Faithfulness	Is this consistent with the source?
Groundedness	Can every claim be traced to evidence?

💡 Like grading a student's cited essay on four separate axes — is the conclusion correct, does it answer the actual prompt, does it accurately represent the sources it cites, and does every claim actually have a citation backing it?

EVERYDAY EXAMPLE

A RAG answer could be factually accurate (correct by coincidence) yet not grounded (the retrieved documents didn't actually support that specific claim) — a distinction that matters enormously for trust (Module 10, 3.3).

WHY IT MATTERS

Optimizing for the wrong metric gives a false sense of quality — a system can score well on relevance while quietly failing on groundedness, which is exactly the kind of gap that causes real-world trust problems.

COMMON MISTAKES

Measuring only accuracy and assuming relevance, faithfulness, and groundedness will automatically follow — they don't always correlate.

BEST PRACTICES

Track multiple relevant metrics together for any non-trivial system, especially groundedness and faithfulness for anything RAG-based (Section 3.8).

SUMMARY

Accuracy, relevance, faithfulness, and groundedness are distinct, sometimes-diverging quality dimensions — track the ones that matter for your specific task, not just one.

🧑‍🔬 3.7 Hallucination Evaluation

DEFINITION

Hallucination evaluation is the systematic measurement of how often and how severely a system produces confidently false or unsupported content (Module 01, 3.11), rather than relying on anecdotal spot-checks.

SIMPLE EXPLANATION

This typically combines groundedness checking (Section 3.6) with fact-verification — either an LLM-as-a-judge (Section 3.3) checking claims against source material, or specialized hallucination-detection methods — producing a measurable hallucination rate.

💡 *Like a newsroom running a formal fact-check pass on every article and tracking an error rate over time, rather than only noticing mistakes when a reader complains.*

EVERYDAY EXAMPLE

Running a RAG system's outputs through a groundedness judge that flags any claim not traceable to a retrieved chunk, then tracking that flagged-claim rate over each release.

WHY IT MATTERS

In high-stakes domains, an unmeasured hallucination rate means you genuinely don't know how often your system is confidently wrong — measurement is the prerequisite for improvement (Module 10, 3.3).

COMMON MISTAKES

Only checking for hallucinations reactively (after a user complaint) instead of proactively measuring a rate on an ongoing basis.

BEST PRACTICES

Build hallucination checking into your regular evaluation pipeline (Section 3.10), not just an occasional manual audit.

SUMMARY

Hallucination evaluation turns an anecdotal concern into a tracked, measurable rate — essential for any system where factual reliability matters.

🧑‍🔬 3.8 RAG Evaluation

DEFINITION

RAG evaluation specifically measures the quality of a retrieval-augmented generation system's two separate stages — retrieval quality (did it find the right chunks, Module 07) and generation quality (did it answer well given those chunks).

SIMPLE EXPLANATION

A RAG system can fail in two independent ways: bad retrieval (the right information was never found) or bad generation (the right information was found but poorly used) — evaluating them separately tells you exactly where to focus fixes.

Retrieval Quality

(right chunks found?)

+

Generation Quality

(used well?)

=

Overall RAG Quality

💡 *Like evaluating a research assistant on two separate skills — did they find the right documents in the archive, and separately, did they write a good summary from what they found? A weak summary from great documents is a different problem than a great summary from the wrong documents.*

EVERYDAY EXAMPLE

Measuring "percentage of queries where the correct chunk was in the top-3 retrieved results" (retrieval) separately from "percentage of answers judged faithful to the retrieved chunks" (generation).

WHY IT MATTERS

Without splitting these two stages, a bad overall RAG score gives you no idea whether to fix chunking/retrieval (Module 07, 3.2, 3.5) or prompt/generation — very different fixes.

COMMON MISTAKES

Evaluating only the final answer's quality, without separately checking whether retrieval actually surfaced the right content in the first place.

BEST PRACTICES

Always evaluate retrieval and generation quality separately for a RAG system, alongside groundedness (Section 3.6) for the generation stage specifically.

SUMMARY

RAG evaluation splits quality measurement into retrieval and generation stages — critical for correctly diagnosing where a RAG system is actually failing.

3.9 Agent Evaluation

DEFINITION

Agent evaluation measures the quality of a multi-step agent's (Module 08) behavior across an entire task — not just the final answer, but the path it took to get there: tool choices, step efficiency, and error recovery.

SIMPLE EXPLANATION

Since agents make many decisions per task (Module 08, 3.6's loop), evaluation needs to look beyond "did it get the right final answer" to also ask: did it use the right tools, did it take a reasonable number of steps, and did it recover sensibly from errors along the way.

 *Like evaluating a new employee not just on whether they eventually finished a project, but also on whether they took a sensible approach, used the right resources, and recovered well when something didn't go as planned.*

EVERYDAY EXAMPLE

Evaluating a research agent (Module 14, 3.4) not just on its final report's accuracy, but also on whether it called the search tool efficiently (not redundantly, Module 08, 3.3) and correctly cited its sources.

WHY IT MATTERS

An agent can reach a correct final answer through an inefficient, expensive, or fragile path — evaluating only the end result misses real reliability and cost problems (Module 09, 3.4).

COMMON MISTAKES

Evaluating agents purely on final-answer correctness, missing step-level inefficiencies, redundant tool calls, or fragile error handling.

BEST PRACTICES

Evaluate both the final outcome and the process: step count, tool selection accuracy (Module 06, 3.5), and error recovery behavior.

SUMMARY

Agent evaluation looks at the whole path, not just the destination — essential given how much can go right or wrong across a multi-step agent loop.

3.10 Prompt Regression Testing

DEFINITION

Prompt regression testing is re-running your evaluation suite automatically whenever a prompt (or model, or pipeline) changes, to catch quality drops before they reach production.

SIMPLE EXPLANATION

This directly extends Module 09's versioning and evaluation cycle into an automated, continuous practice — the AI-engineering equivalent of a CI/CD test suite that runs on every code change in traditional software.

💡 *Like an automated test suite that runs every time a developer pushes new code — catching a bug immediately, rather than discovering it days later in production.*

EVERYDAY EXAMPLE

Automatically re-running your full evaluation dataset (Section 3.2) against every new prompt version before it's allowed to merge, blocking the merge if scores drop below a threshold.

WHY IT MATTERS

Without regression testing, a prompt "improvement" that quietly breaks a different case can slip into production unnoticed until a user hits it.

COMMON MISTAKES

Only evaluating manually and occasionally, rather than automatically on every meaningful change — regressions slip through the gaps between manual checks.

BEST PRACTICES

Automate evaluation to run on every prompt/model/pipeline change; set a clear quality threshold that blocks a change from shipping if scores regress.

SUMMARY

Prompt regression testing automates Module 09's evaluation cycle into a continuous safety net — catching quality drops before users do.

🔗 3.11 Error Analysis

DEFINITION

Error analysis is the structured process of examining failed evaluation cases in depth to find patterns — categorizing why the system failed, not just measuring that it failed.

SIMPLE EXPLANATION

A score alone ("73% accuracy") tells you how often something fails, not why — error analysis groups the failing cases by root cause (retrieval miss, misunderstood instruction, hallucination, edge case) to reveal what's actually worth fixing first.

💡 *Like a doctor not just noting "patient still has symptoms" but investigating exactly which underlying cause is responsible — the fix is completely different depending on the root cause.*

EVERYDAY EXAMPLE

Reviewing 30 failed RAG evaluation cases and finding 20 were retrieval failures (wrong chunks) and 10 were generation failures (right chunks, poor answer) — pointing clearly toward fixing retrieval first.

WHY IT MATTERS

Error analysis is what turns an evaluation score into an actionable engineering roadmap — without it, you know something is wrong but not what to fix first.

COMMON MISTAKES

Treating a low evaluation score as the end of the investigation, rather than the start of a deeper look into why cases are failing.

BEST PRACTICES

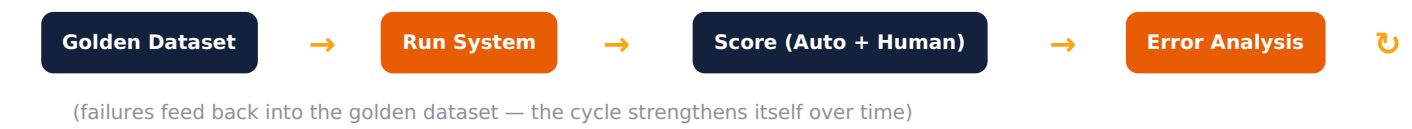
Categorize failures by root cause after every evaluation run; feed the most valuable/tricky failing cases back into your golden dataset (Section 3.2) so they're never missed again.

SUMMARY

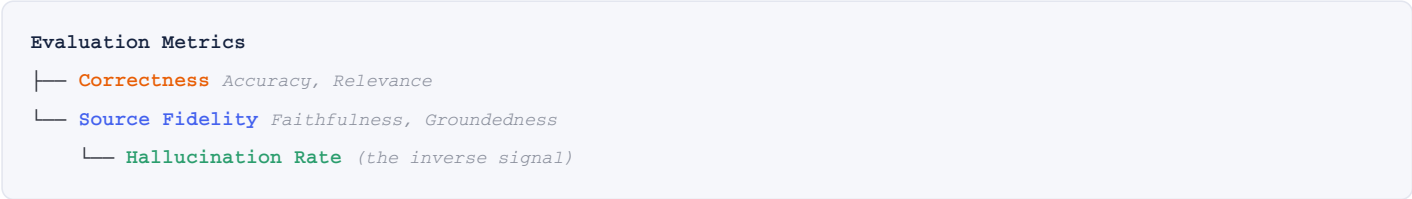
Error analysis turns raw failure counts into categorized root causes — the step that makes evaluation results genuinely actionable.

4. Visual Learning

🔄The Full Evaluation Cycle



🌳Metric Family Tree



📊RAG vs. Agent Evaluation Focus

System Type	What Gets Evaluated Separately
RAG	Retrieval quality + Generation quality
Agent	Final outcome + Step-by-step process (tool use, efficiency, recovery)

5. Real World Applications

Product	How these fundamentals show up
Enterprise RAG deployments	Separate retrieval/generation metrics tracked per release, with groundedness gating production launches.
Customer support AI	LLM-as-a-judge scores tone/empathy at scale; human evaluators spot-check weekly samples.
Coding agents	Agent evaluation tracks task completion rate, step efficiency, and test-pass rate as core metrics.
CI/CD for AI features	Prompt regression testing blocks merges when evaluation scores drop below threshold.

6. Practical Examples

Beginner: Building a tiny golden dataset

Pick a task from earlier in this course and write 10 test cases with carefully verified expected answers — your first golden dataset.

Beginner: Spotting the metric gap

Write one example answer that would score well on relevance but poorly on groundedness, to feel the difference between the two metrics concretely.

Intermediate: LLM-as-a-judge rubric

Write a judge-LLM prompt with a clear 1-5 rubric for scoring "helpfulness" of a customer support reply, including one calibration example for each score level.

Advanced: RAG failure diagnosis

Given 10 hypothetical failed RAG answers, categorize each as a retrieval failure or a generation failure, and explain your reasoning for 3 of them.

7. Code Examples

🔗Python — LLM-as-a-judge evaluation harness

```
# pip install anthropic
import anthropic
import json

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

JUDGE_RUBRIC = """Score the response from 1-5 on FAITHFULNESS to the source:
5 = every claim is directly supported by the source
3 = mostly supported, minor unsupported details
1 = contains claims not supported by the source at all

Return ONLY JSON: {"score": , "reasoning": ""}"""

def judge_response(source, response):
    prompt = f"""{JUDGE_RUBRIC}

<source>
{source}
</source>

<response>
{response}
</response>"""

    result = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=150, temperature=0,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(result.content[0].text)

# Run across a golden dataset and aggregate
golden_dataset = [
    {"source": "Refunds accepted within 30 days.", "response": "You can get a refund within 30 days."},
    {"source": "Refunds accepted within 30 days.", "response": "Refunds are accepted within 60 days."}, # bad
]

scores = [judge_response(c["source"], c["response"])["score"] for c in golden_dataset]
print(f"Average faithfulness: {sum(scores)/len(scores):.1f}/5")
```

📄JavaScript — Simple error analysis categorizer

```
// Categorize failed evaluation cases by likely root cause
function categorizeFailure(testCase, result) {
    if (!result.retrievedRelevantChunk) return "retrieval_failure";
    if (result.groundednessScore < 3) return "generation_ungrounded";
    if (result.score < 3) return "generation_quality";
    return "uncategorized";
}

const failedCases = [
    { id: 1, retrievedRelevantChunk: false, groundednessScore: 2, score: 2 },
    { id: 2, retrievedRelevantChunk: true, groundednessScore: 4, score: 2 },
];

const categorized = failedCases.map((c) => ({
    id: c.id,
    category: categorizeFailure(c, c),
}));

// Tally by category to see what to fix first
const tally = {};
categorized.forEach((c) => { tally[c.category] = (tally[c.category] || 0) + 1; });
console.log(tally); // e.g. { retrieval_failure: 1, generation_quality: 1 }
```



Tip

Store every evaluation run's results (not just the summary score) — that raw data is exactly what Section 3.11's error analysis needs to work with later.

8. Common Mistakes

- **Using an unverified dataset as if it were golden-quality**, producing misleading evaluation scores.
- **Trusting LLM-as-a-judge scores blindly** without periodic human calibration.
- **Relying on only automated or only human evaluation**, missing the gap the other would have caught.
- **Measuring only accuracy** and assuming relevance, faithfulness, and groundedness follow automatically.
- **Evaluating a RAG system's final answer only**, without separating retrieval and generation quality.
- **Evaluating agents purely on final-answer correctness**, missing step-level inefficiency or fragile error handling.
- **Stopping at a low score** without doing error analysis to find the actual root cause.

9. Best Practices

- **Invest in a small, verified golden dataset** before a large, unverified one.
- **Combine LLM-as-a-judge with periodic human spot-checks** for calibration.
- **Track multiple relevant metrics together**, especially groundedness for RAG-based systems.
- **Evaluate RAG retrieval and generation stages separately** to diagnose failures correctly.
- **Evaluate agents on process, not just outcome** — step efficiency and tool selection matter too.
- **Automate regression testing** on every prompt/model/pipeline change.
- **Always follow a low score with error analysis**, and feed found failures back into your golden dataset.

10. Hands-on Activities

Easy 5 exercises

1. Write 10 test cases with verified answers for a task from earlier in this course.
2. Define, in your own words, the difference between accuracy and groundedness with an original example.
3. Write a simple LLM-as-a-judge rubric (1-3 criteria) for a task of your choice.
4. List 2 scenarios where automated evaluation is sufficient, and 2 where human evaluation is necessary.
5. Sketch the evaluation cycle diagram from Section 4 from memory.

Intermediate 3 exercises

1. Build the Python LLM-as-a-judge harness from Section 7 and run it against 10 real examples.
2. Take a RAG project from Module 07/14 and evaluate retrieval and generation quality separately.
3. Run error analysis on 10 failed cases from any evaluation you've done in this course, categorizing each by root cause.

Challenge 2 exercises

1. Design a full agent evaluation rubric (Section 3.9) covering both outcome and process for an agent from Module 08/14.
2. Set up automated prompt regression testing (Section 3.10) for a real prompt you've built, with a defined quality threshold that blocks changes.

11. Mini Project

Goal

Build a complete evaluation pipeline for one of Module 14's real projects — golden dataset, automated scoring, and error analysis, end to end.

Requirements

- An Anthropic API key
- Python or JavaScript
- A project from Module 14 (Chat with PDF is a strong choice, given its clear right/wrong answers)

Step-by-Step Instructions

1. Build a 15-20 case golden dataset with verified expected answers.
2. Build an LLM-as-a-judge scorer for faithfulness/groundedness (Section 7).
3. Run your project against the full dataset and record all results (not just the average).
4. Categorize every failing case by root cause (Section 3.11).
5. Write a short report: overall score, breakdown by failure category, and your top 2 recommended fixes.

Expected Output

```
Golden dataset: 18 cases
Overall faithfulness score: 4.1/5

Failure breakdown (4 failing cases):
- retrieval_failure: 3
- generation_ungrounded: 1

Recommendation: Prioritize retrieval/chunking fixes (Module 07, 3.2)
before touching the generation prompt.
```

Possible Improvements

- Set up automated regression testing so this evaluation runs every time you change the project's prompt.
- Add human evaluation on a 5-case sample to calibrate your LLM-as-a-judge scores.
- Grow the golden dataset over time as you find new important edge cases.

12. Review Questions

Multiple Choice (10)

1. A golden dataset is best described as:

- A) Any random dataset B) A carefully verified, high-quality reference dataset C) A type of model D) An unverified auto-generated set

2. LLM-as-a-judge is used primarily to:

- A) Train new models B) Score outputs against criteria using another LLM call C) Replace all human evaluation permanently D) Reduce token cost to zero

3. Groundedness measures:

- A) How fast a response is B) Whether every claim can be traced to supporting evidence C) Token count D) Model size

4. RAG evaluation should separately assess:

- A) Only the final answer B) Retrieval quality and generation quality C) Only the model's price D) Only latency

5. Agent evaluation should consider:

- A) Only the final outcome B) Both the final outcome and the step-by-step process C) Nothing beyond cost D) Only tool names

6. Prompt regression testing is best described as:

- A) A one-time manual check B) Automated re-evaluation on every prompt/model change C) Ignoring evaluation after launch D) A type of fine-tuning

7. Error analysis primarily helps by:

- A) Hiding failures B) Categorizing failures by root cause to guide fixes C) Increasing temperature D) Replacing evaluation entirely

8. Benchmarking requires which condition to be meaningful?

- A) Changing the dataset each time B) Keeping dataset and method fixed while comparing systems C) No dataset at all D) Only human evaluation

9. Why combine automated and human evaluation?

- A) They're identical, so it doesn't matter B) Each catches different kinds of problems the other misses C) It's required by every provider D) It's always more expensive with no benefit

10. A response can be relevant but not accurate. This illustrates:

- A) Metrics are all the same thing B) Different evaluation metrics can diverge and should be tracked separately C) Relevance is useless D) Accuracy doesn't matter

Identification (5)

1. _____ is using a separate LLM call to score another LLM's output against criteria.

2. _____ measures whether every claim in a response can be traced to supporting evidence.

3. _____ automatically re-runs evaluation whenever a prompt or model changes.

4. _____ is the structured process of categorizing failed cases by root cause.

5. _____ is comparing evaluation results against a fixed reference point like a previous version.

True or False (5)

1. LLM-as-a-judge scores should be trusted permanently without any human calibration.

2. RAG evaluation should separate retrieval quality from generation quality.

3. Agent evaluation should only consider the final answer, never the process.

4. Benchmarking requires keeping the dataset and method fixed while comparing systems.

5. Error analysis should follow a low evaluation score to find the actual root cause.

Situational (5)

Situational (5)

- 1. Your RAG system scores poorly overall. How would you determine whether to fix retrieval or generation first?**
- 2. You want to evaluate tone/empathy across 1,000 support replies without reading all of them. What method fits?**
- 3. A prompt change was merged and users started complaining about quality. What practice would have caught this before release?**
- 4. Your agent reaches correct final answers but uses 3x more tool calls than necessary. What evaluation dimension is this?**
- 5. You have a low evaluation score but don't know why. What's the next step?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-LLM-as-a-judge, 2-Groundedness, 3-Prompt regression testing, 4-Error analysis, 5-Benchmarking

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Evaluate retrieval and generation quality separately to see which stage is actually failing.
2. LLM-as-a-judge, scaled with periodic human calibration.
3. Automated prompt regression testing before the change was merged.
4. Agent evaluation's process dimension — step efficiency, not just final-outcome correctness.
5. Run error analysis — categorize the failing cases by root cause.

13. Cheat Sheet

Evaluation Basics

Data + Method + Metric, all deliberate. Golden dataset = verified, trustworthy reference.

Metrics

Accuracy, Relevance, Faithfulness, Groundedness — track together, they can diverge.

Methods

LLM-as-a-judge — scalable, needs calibration.

Automated + Human — combine both.

Keep It Going

Regression test on every change; error-analyze every failure; feed fixes back into golden dataset.

14. Key Takeaways

- Every evaluation system needs three deliberate choices: data, method, and metric.
- Golden datasets are small, carefully verified reference sets — trustworthy over large, unverified ones.
- LLM-as-a-judge scales subjective evaluation but needs periodic human calibration to stay trustworthy.
- Accuracy, relevance, faithfulness, and groundedness are distinct metrics that can diverge — track them together.
- RAG evaluation must separate retrieval quality from generation quality to correctly diagnose failures.
- Agent evaluation must consider the full step-by-step process, not just the final outcome.
- Prompt regression testing automates evaluation on every change — the AI-engineering equivalent of a CI/CD test suite.
- Error analysis turns a raw score into an actionable roadmap by categorizing failures by root cause.

15. Additional Resources

Official Documentation

- Anthropic — evaluation and testing guidance (docs.claude.com)

Tools

- Promptfoo, LangSmith, Braintrust, and similar open-source/commercial evaluation frameworks for larger-scale eval pipelines.

Related Modules

- Module 09 — Prompt Optimization (the foundational testing/evaluation concepts this module extends)
- Module 07 — RAG, Module 08 — AI Agents (the systems this module's specialized evaluation targets)

Communities & Further Learning

- AI evaluation research and engineering blogs — this subfield is evolving quickly with new methods for judge calibration and automated grading.